数据结构与算法分析

人工智能与自动化学院 谭毅华

yhtan@hust.edu.cn 87556301

12. 回溯算法

- 回溯法的深度优先搜索策略
- 回溯法解题的算法框架
  - （1）递归回溯
  - （2）迭代回溯
  - （3）子集树算法框架
  - （4）排列树算法框架
- 通过例子学习回溯法的设计策略
  - （1）装载问题　（2）批处理作业调度
  - （3）符号三角形问题　（4） n皇后问题
  - （5）最大团问题　（6）图的m着色问题
  - （7）旅行售货员问题　（8）圆排列问题
  - （9）连续邮资问题

回溯法简介

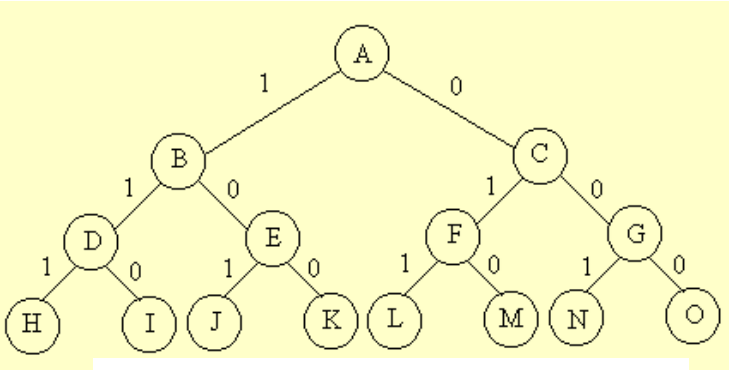
- 有许多问题，当需要找出它的解集或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法
- 基本做法是搜索
  - 组织得井井有条的，能避免不必要搜索的穷举式搜索法
  - 适用于解一些组合数相当大的问题
- 在问题的解空间树中，按深度优先策略，从根结点出发搜索解空间树
  - 在解空间树的任意结点，先判断该结点是否包含问题的解
  - 若肯定不包含，则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树按深度优先策略搜索

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

问题的解空间

- 问题的解向量：回溯法希望一个问题的解能够表示成一个n元式$(x_1,x_2,\ldots,x_n)$的形式。例如0-1背包问题
  - 显约束：对分量x_i的取值限定。（例如取1或0）
  - 隐约束：为满足问题的解而对不同分量之间施加的约束。（例如总重量约束）
  - 解空间：对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。

注意其组织结构：同一个问题可以有多种表示，有些表示方法更简单，所需表示的状态空间更小（存储量少，搜索方法简单）。



n=3时的0-1背包问题用完全二叉树表示的解空间。

華中科技大學 人工智能与自动化学院

二叉树解空间结点类型

- 扩展结点
  - 一个正在产生儿子的结点称为扩展结点
- 活结点
  - 一个自身已生成但其儿子还没有全部生成的节点
- 死结点
  - 一个所有儿子已经产生的结点

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

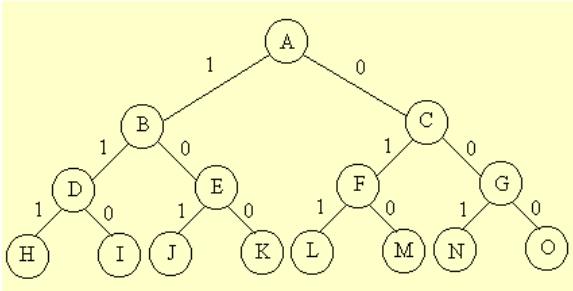
生成问题状态的基本方法

- 深度优先的问题状态生成法
  - 如果对一个扩展结点R，一旦产生了它的一个儿子C，就把C当做新的扩展结点。在完成对子树C（以C为根的子树）的穷尽搜索之后，将R重新变成扩展结点，继续生成R的另一个儿子（如果存在）
- 宽度优先的问题状态生成法
  - 在一个扩展结点变成死结点之前，它一直是扩展结点
- 回溯法
  - 为了避免生成那些不可能产生最佳解的问题状态，要不断地利用限界函数 (bounding function) 来处死那些实际上不可能产生所需解的活结点，以减少问题的计算量
  - 具有限界函数的深度优先生成法称为回溯法

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

回溯法的例子

- n=3时的0-1背包问题：重量w=[16, 15, 15]， 价值p=[45, 25, 25]， 容量c=30，剩余容量r. 过程描述如下：
  - 根节点是唯一的活结点，也是当前的扩展结点
  - 可扩展至B或C，若按顺序先扩展至B，即选了w1，此时背包容量剩余r=14，获取价值为45.



  - 从结点B可以扩展至D或E，而D是不可行解，E点则不需扩展容量，故E为扩展结点。此时A,B,E为活结点。结点E处，r=14, 价值为45.
  - 从结点E又可扩展至J或K，而J为不可行解，K则可行。由于K为叶子结点，可得到一个可行解$x=(1,0,0)$，并由根结点到叶子结点的路径确定。K成为死结点。
  - 返回到E，则E为死结点。再返回到B，B也为死结点。在返回A，A成为扩展结点。扩展到C，此时r=30。先扩展到F，则r=15,价值为25。继续可扩展到叶子结点L，r=0，价值=50。形成了可行解$x=(0,1,1)$且价值高于刚才找到的可行解。
  - 按此方式可以继续搜索，但$x=(0,1,1)$是最好的解。

華中科技大學 人工智能与自动化学院

回溯法的基本思想

- 解空间
  - 针对所给问题，定义问题的解空间；
  - 确定易于搜索的解空间结构；
- 搜索
  - 以深度优先方式搜索解空间；
  - 在搜索过程中用剪枝函数避免无效搜索。

常用剪枝函数

- 用约束函数在扩展结点处剪去不满足约束的子树；
- 用限界函数剪去得不到最优解的子树。

華中科技大學 人工智能与自动化学院

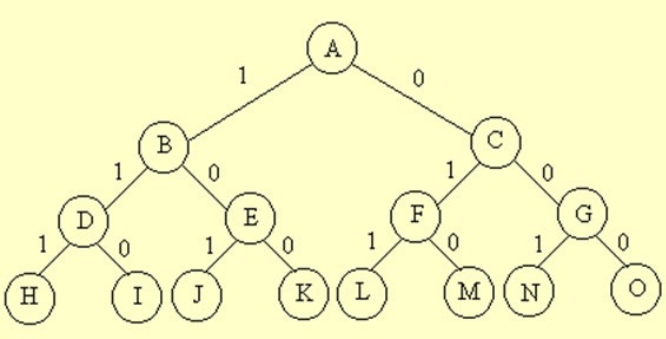
回溯法的基本思想

- 用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间
  - 在任何时刻，算法只保存从根结点到当前扩展结点的路径；
  - 如果解空间树中从根结点到叶结点的最长路径的长度为h(n)，则回溯法所需的计算时间通常为O(h(n))；
  - 而显式地存储整个解空间则需要$O(2^{h(n)})$或O(h(n)!)内存空间。

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

回溯法的算法框架–递归回溯

- 回溯法对解空间作深度优先搜索，因此，在一般情况下用递归方法实现回溯法




```
void backtrack (int t)
{
    if (t>n) output(x); //叶子结点
    else  //当前扩展节点的子树的起始f(n,t)/终止孩子g(n,t)
      for (int i=f(n,t);i<=g(n,t);i++) {
          x[t]=h(i); //解赋值为第i个可选值h(i)
          if (constraint(t)&&bound(t)) //剪枝函数：取值约束与限界
              backtrack(t+1);          //进入子节点
      }
}
```

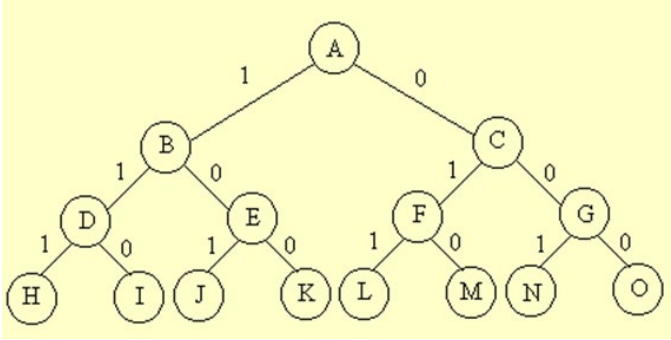
调用1次backtrack(1)即可完成回溯求解

華中科技大學 人工智能与自动化学院

回溯法的算法框架–迭代回溯

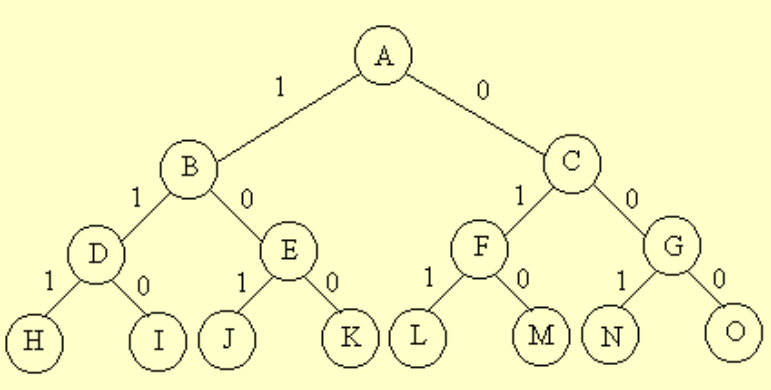
- 采用树的非递归深度优先遍历算法，可将回溯法表示为一个非递归迭代过程

```
void iterativeBacktrack (){//n为总深度，t为当前层
  int t=1;
  while (t>0) {
    if (f(n,t)<=g(n,t)) {//当前未搜索过子树的起始与终止编号
      for (int i=f(n,t);i<=g(n,t);i++) {
        x[t]=h(i);    //取扩展结点的第i个值，即第i个子树
        if (constraint(t)&&bound(t)) {
          if (solution(t))  output(x); //若得到可行解
          else t++; //进到树的下一层
        }
      }
    }
    else t--; //对于死结点，回溯
  }
}
```



回溯法的算法框架–子集树

- 当所给问题是从n个元素集合S中找出满足某种性质的子集时，相应的解空间树称为子集树。
- 子集树通常有2^n个叶子结点,结点总个数为$2^{n+1}-1$。
- 例如0-1背包问题的解空间。



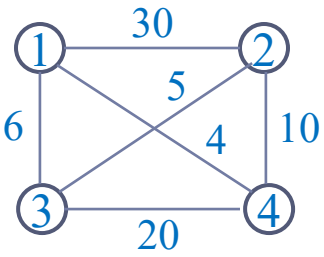
遍历子集树需$O(2^n)$计算时间

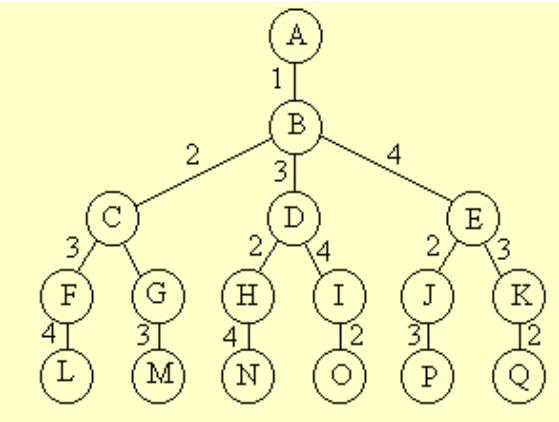
```
void backtrack (int t){
  if (t>n) output(x);
  else
      for (int i=0; i<=1; i++) {
        x[t]=i;
        if (constraint(t)&&bound(t))
           backtrack(t+1);
      }
}
```


華中科技大學 人工智能与自动化学院

回溯法的算法框架–排列树

- 当所给问题是确定n个元素满足某种排列性质时，相应的解空间树称为排列树。
- 排列树通常有n!个叶子结点。
- 例如旅行售货员问题的解空间。





问题：售货员要到若干城市去推销商品，已知各城市之间的旅费。他要选定一条从驻地出发，经过所有城市一遍，最后回到驻地的路线，使得总费用最小。

遍历排列树需要O(n!)计算时间

```
void backtrack (int t){
  if (t>n) output(x);
  else
     for (int i=t; i<=n; i++) {
        swap(x[t], x[i]);
        if (constraint(t)&&bound(t))
           backtrack(t+1);
        swap(x[t], x[i]);
     }
}
```

调用1次backtrack(1)即可完成回溯求解对x先初始化为单位排列(1,2,…,n).

范例1–装载问题

- 有一批共n个集装箱要装上2艘载重量分别为c_1和c_2的轮船，其中集装箱i的重量为w_i，且

$$\sum_{i=1}^{n} w_i \le c_1 + c_2$$

- 装载问题要求确定是否有一个合理的装载方案可将这 n 个集装箱装上这2艘轮船？如果有，找出一种装载方案。
- 容易证明，如果一个给定装载问题有解，则采用下面的策略可得到最优装载方案：

(1) 首先将第一艘轮船尽可能装满；
(2) 将剩余的集装箱装上第二艘轮船。

華中科技大學 人工智能与自动化学院

范例1-装载问题

- 将第一艘轮船尽可能装满等价于选取全体集装箱的一个子集，使该子集中集装箱重量之和最接近第一艘轮船的载重量。由此可知，**装载问题等价于特殊的0-1背包问题：**

用回溯法设计解装载问题的$O(2^n)$计算时间算法。在某些情况下该算法优于动态规划算法。

$$\max \sum_{i=1}^{n} w_i x_i$$

$$\text{s.t.} \sum_{i=1}^{n} w_i x_i \le c_1 \quad x_i \in \{0,1\}, 1 \le i \le n$$

- 解空间：子集树　　第j+1层的结点Z处，当前载重量：$cw = \sum_{i=1}^{j} w_i x_i$
- 可行性约束函数(选择当前元素)：

$$\sum_{i=1}^{j} w_i x_i \le c_1$$

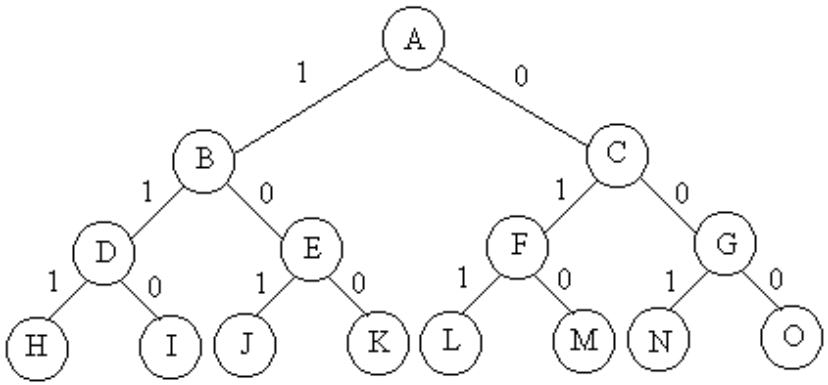
- 上界函数(不选择当前元素)：
  当前载重量cw + 剩余集装箱的重量r ≤ 当前最优载重量bestw

不断变化取优

華中科技大學 人工智能与自动化学院

范例1–装载问题

➢ 算法实现




```
void backtrack (int i)
  {// 搜索第i层结点
    if (i > n)  {// 到达叶子结点
        bestw = cw; return;}
    r -= w[i];   //剩余集装箱的重量
    if (cw + w[i] <= c) {// 搜索左子树
      x[i] = 1;
      cw += w[i];
      backtrack(i + 1);
      cw -= w[i];
    }
    if (cw + r > bestw)  {// 搜索右子树
      x[i] = 0;
      backtrack(i + 1);      //如果不满足可行解比当前装载的重量更重，则直接截断右子树
    }
    r += w[i];
  }
```

- 问题1：bestw没有比较大小，就直接覆盖了，有没有问题？
- 问题2：为什么cw先加w[i]，又减掉。
- 问题3：左子树和右子树有什么区别？

華中科技大學 人工智能与自动化学院

范例2-批处理作业调度

- 问题：给定n个作业的集合$\{J_1,J_2,\ldots,J_n\}$。
  - 每个作业必须先由机器1处理，然后由机器2处理
  - 作业J_i需要机器j的处理时间为t_{ji}
  - 对于一个确定的作业调度，设F_{ji}是作业i在机器j上完成处理的时间。
  - 所有作业在机器2上完成处理的时间和称为**该作业调度的完成时间和**。

$$f=\sum_{i=1}^{n}F_{2i}$$

- 批处理作业调度问题要求对于给定的n个作业，制定**最佳作业调度方案**，使其**完成时间和达到最小**。

華中科技大學 人工智能与自动化学院

范例2-批处理作业调度

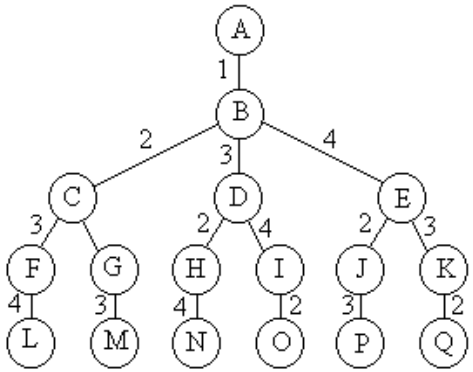
t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

- 这3个作业的6种可能的调度方案
  - 1,2,3；1,3,2；2,1,3；2,3,1；3,1,2；3,2,1；
  - 它们所相应的完成时间和分别是19，18，20，21，19，19。
- 易见，最佳调度方案是1,3,2，其完成时间和为18。

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

范例2-批处理作业调度

➢ 解空间




```
class Flowshop {
  friend Flow(int**, int, int []);
  private:
    void Backtrack(int i);
    int  **M,   // 各作业所需的处理时间
         *x,   // 当前作业调度
     *bestx,   // 当前最优作业调度
       *f2,   // 机器2完成处理时间
        f1,   // 机器1完成处理时间
         f,   // 完成时间和
     bestf,   // 当前最优值
         n;   // 作业数};
```

```
void Flowshop::Backtrack(int i)
{
  if (i > n) {
     for (int j = 1; j <= n; j++)
       bestx[j] = x[j];
     bestf = f;
     }
  else
    for (int j = i; j <= n; j++) {
      f1+=M[x[j]][1];
      f2[i]=((f2[i-1]>f1)?f2[i-1]:f1)+M[x[j]][2];
      f+=f2[i];
      if (f < bestf) {
        Swap(x[i], x[j]);
        Backtrack(i+1); //得到一个排列
        Swap(x[i], x[j]); //交换回来，维持字典顺序
        }
      f1- =M[x[j]][1];
      f- =f2[i];
      }
}
```

回溯，解除当前结点的加入

華中科技大學 人工智能与自动化学院

范例3-符号三角形问题

下图是由 14 个“+”和 14 个“-”组成的符号三角形，2 个同 号下面都是“+”，2 个异号下面都是“-”。

+ + - + - + +
+ - - - - +
- + + + -
- + + -
- + -
- -
+

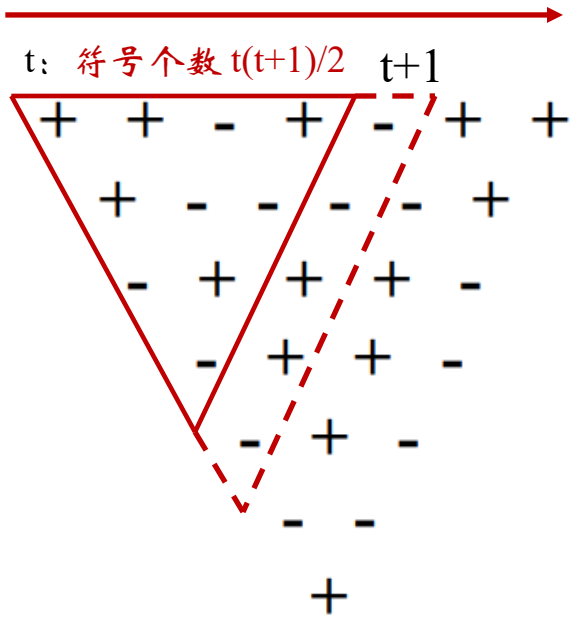
- 在一般情况下，符号三角形的第一行有 n 个符号。
- 符号三角形问题要求对于给定的 n，计算有多少个不同的符号三角形，使其所含的“+”和“-”的个数相同。

華中科技大學 人工智能与自动化学院

范例3-符号三角形问题

算法设计

- 解向量：用 n 元组 x[1:n] 表示符号三角形的第一行。
- 可行性约束函数：当前符号三角形所包含的“+”个数与“-”个数均不超过 n*(n+1)/4。
- **给定n时无解的判断**：n*(n+1)/2 为奇数



范例3-符号三角形问题

```
Class Triangle
{   friend int compute(int);
 private:
    void Backtrack(int t);
     int n,          //第一行符号个数
          half,     //n*(n+1)/4
          count, // 当前“+”个数
          **p,   //符号三角形矩阵
     long sum; //三角形个数
};
```

复杂度分析

- 计算可行性约束需要O(n) 时间，在最坏情况下有$O(2^n)$个结点需要计算可行性约束，故解符号三角形问题的回溯算法所需的计算时间为 $O(n2^n)$

```
void Triangle::Backtrack(int t)
{//约束条件
    if ((count>half)||(t*(t+1)/2-count>half))
        return;
    if (t>n) sum++;
    else
        for (int i=0;i<2;i++) {
            p[1][t]=i;
            count+=i;   //1为+号
            for (int j=2;j<=t;j++) {//增加斜行
              p[j][t-j+1]=p[j-1][t-j+1]^p[j-1][t-j+2];
              count+=p[j][t-j+1];//可行性判断的关
                                    键计数器 O(n)
            }
            Backtrack(t+1);
            for (int j=2;j<=t;j++)
                count-=p[j][t-j+1];
            count-=i;
        }
}
```

回溯，解除当前结点的加入

華中科技大學 人工智能与自动化学院

范例4 – n皇后问题

- 在n×n格的棋盘上放置彼此不受攻击的n个皇后。按照国际象棋的规则，皇后可以攻击与之在同一行或同一列或同一斜线上的棋子。n后问题等价于在n×n格的棋盘上放置n个皇后，任何2个皇后不放在同一行或同一列或同一斜线上。

	1	2	3	4	5	6	7	8
1				Q				
2						Q		
3								Q
4		Q						
5							Q	
6	Q							
7			Q					
8					Q			

華中科技大學 人工智能与自动化学院

范例4 – n皇后问题

- 解向量：$(x_1, x_2, \ldots, x_n)$
  - x_i表示皇后位于第i行第x_i列
- 显约束：$x_i=1,2,\ldots,n$
- 隐约束：
  1)不同列：$x_i \neq x_j$
  2)不处于同一正、反对角线：$|i-j| \neq |x_i-x_j|$

```
class Queen {
   friend int nQueen();
 private:
   bool Place(int k);  //检查其可行性
   void Backtrack(int k);
    int n,    // 皇后数量
        *x,   // 当前解
    long sum; //当前已找到的可行方案数
};
```

```
bool Queen::Place(int k)
{
  for (int j=1;j<k;j++)
    if ((abs(k-j)==abs(x[j]-x[k])) ||
(x[j]==x[k])) return false;
  return true;
}

void Queen::Backtrack(int t)
{
    if (t>n) sum++;
    else
       for (int i=1;i<=n;i++) {
          x[t]=i;
          if (Place(t)) Backtrack(t+1);
       }
 }
```


華中科技大學 人工智能与自动化学院

范例5–最大团问题

给定无向图G=(V，E)。如果U⊆V，且对任意u，v∈U有(u，v)∈E，则称U是G的完全子图。

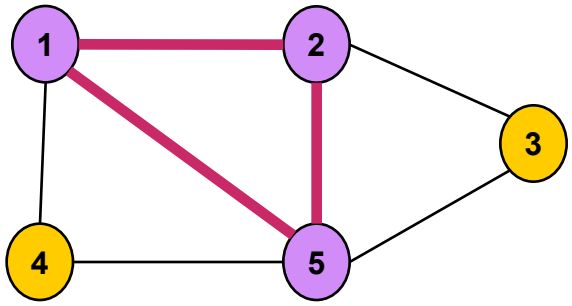
- 团
  - G的完全子图U是G的团当且仅当U不包含在G的更大的完全子图中。
  - G的最大团是指G中所含顶点数最多的团。
- 独立集合
  - 如果U⊆V且对任意u，v∈U有(u，v)∉E，则称U是G的空子图。
  - G的空子图U是G的独立集当且仅当U不包含在G的更大的空子图中。
  - G的最大独立集是G中所含顶点数最多的独立集。

華中科技大學 人工智能与自动化学院

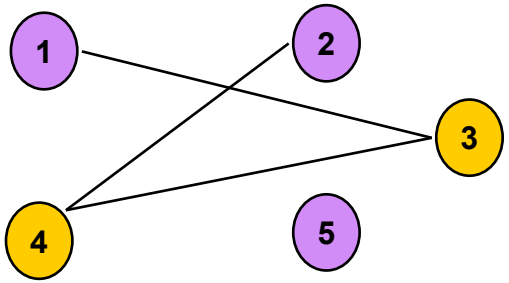
范例5–最大团问题

对于任一无向图G=(V，E)其 补图G'=(V_1，E_1)定义为：V_1=V，且(u，v)∈E_1当且仅当(u，v)∉E。

U是G的最大团当且仅当U是G'的最大独立集。



G=(V，E)

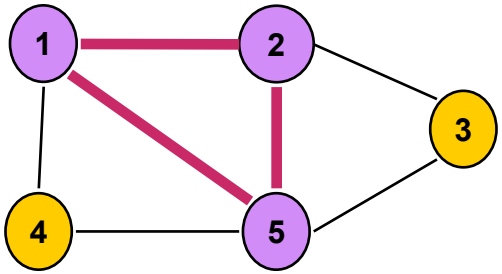


G=(V_1，E_1)

華中科技大学 人工智能与自动化学院

范例5–最大团问题

- 解空间：子集树
- 可行性约束函数：顶点i到已选入的顶点集中每一个顶点都有边相连。
- 上界函数：有足够多的可选择顶点使得算法有可能在右子树中找到更大的团



复杂度分析：
最大团问题的回溯算法backtrack所需的计算时间显然为$O(n2^n)$。

```
void Clique::Backtrack(int i)
{ // 计算最大团
  if (i > n) {// 到达叶结点
    for (int j = 1; j <= n; j++) bestx[j] = x[j];
    bestn = cn;   return;}
  // 检查顶点 i 与当前团的连接
  int OK = 1;
  for (int j = 1; j < i; j++)
    if (x[j] && a[i][j] == 0) {
      // i与j不相连
      OK = 0;  break;}
  if (OK) { // 进入左子树
    x[i] = 1;  cn++;
    Backtrack(i+1);
    x[i] = 0; cn--;}
  if (cn + n - i > bestn) { // 进入右子树
    x[i] = 0;
    Backtrack(i+1);}
}
```


華中科技大學 人工智能与自动化学院

范例5-最大团问题

进一步改进

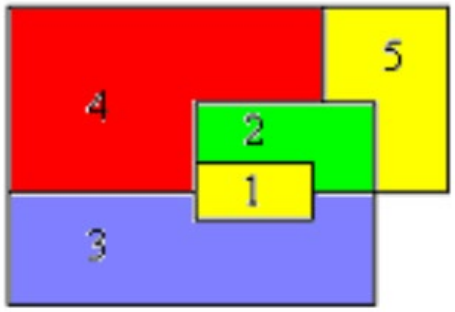
- 选择合适的搜索顺序，可以使得上界函数更有效的发挥作用
  - 例如在搜索之前可以将顶点按度从小到大排序，这在某种意义上相当于给回溯法加入了启发性
- 定义 $Si=\{v_i, v_{i+1},\ldots,v_n\}$，依次求出 $S_n, S_{n-1}, \ldots, S_1$ 的解，从而得到一个更精确的上界函数，若 $cn+S_i<=max$ 则剪枝
  - 同时注意到：从 S_{i+1} 到 S_i，如果找到一个更大的团，那么 v_i 必然属于找到的团，此时有 $S_i=S_{i+1}+1$，否则 $S_i=S_{i+1}$

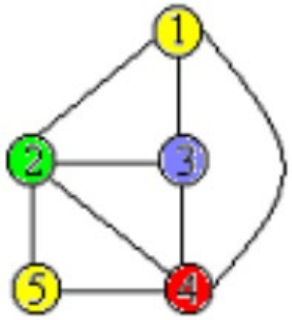
華中科技大學 人工智能与自动化学院

范例6-图的m着色问题

- 给定无向连通图 G 和 m 种不同的颜色，用这些颜色为图 G 的各顶点着色，每个顶点着一种颜色
- 问题模型
  - 是否有一种着色法使 G 中每条边的 2 个顶点着不同颜色，这个问题是图的 m 可着色判定问题
  - 若一个图最少需要 m 种颜色才能使图中每条边连接的 2 个顶点着不同颜色，则称这个数 m 为该图的色数
  - 求一个图的色数 m 的问题称为图的 m 可着色优化问题

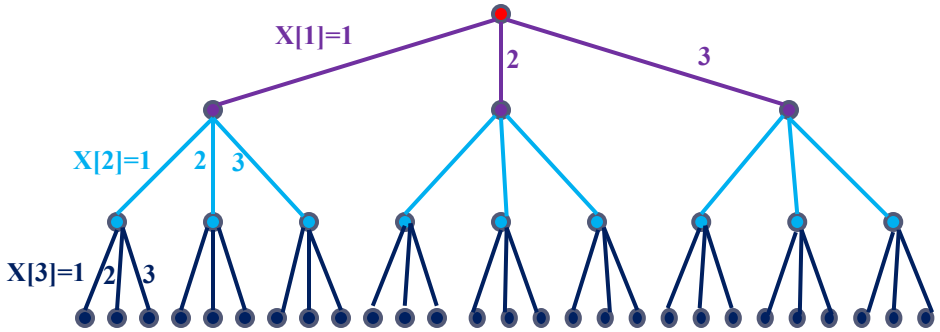
平面图四色猜想：





范例6–图的m着色问题

- 算法设计
  - 用图的邻接矩阵a表示无向连通图G=(V,E)。顶点i的所着颜色用x[i]表示，则问题的解向量为x[1:n]。
  - 解空间：高度为n+1的完全m叉树，n+1层结点为叶结点。
  - 示例：n=3,m=3时的解空间树。



  - Backtrack算法：sum记录当前已找到的可m着色方案。
    - i>n时，算法搜索至叶结点，当前找到的可m着色方案增1
    - i≤ n时，当前扩展结点Z是解空间的内部结点，由OK()函数检查其m个儿子的可行性，并已深度优先方式递归地搜索子树，或剪去不可行子树

華中科技大學 人工智能与自动化学院

范例6-图的m着色问题

```
void Color::Backtrack(int t)
{
   if (t>n) {
     sum++;
     for (int i=1; i<=n; i++)
        cout << x[i] << ' ';
        cout << endl;
   }
   else
     for (int i=1;i<=m;i++) {
       x[t]=i;
       if (Ok(t)) Backtrack(t+1);
     }
}
bool Color::Ok(int k)
{ // 检查颜色可用性
   for (int j=1;j<=n;j++)
     if ((a[k][j]==1)&&(x[j]==x[k])) return false;
   return true;
}
```

复杂度分析

图m可着色问题的解空间树中内结点个数是 $\sum_{i=0}^{n-1} m^i$

对于每一个内结点，在最坏情况下，用 Ok 检查当前扩展结点的每一个儿子所相应的颜色可用性需耗时O(mn)。因此，回溯法总的时间耗费是

$$\sum_{i=0}^{n-1} m^i (mn) = nm(m^n - 1)/(m-1) = O(nm^n)$$

華中科技大學 人工智能与自动化学院

范例7-旅行售货员问题

- 某售货员要到若干城市去推销商品，已知各城市之间的路程(或旅费)，他要选定一条从驻地出发，经过每个城市一次，最后回到驻地的路线，使总的路程(或总旅费)最小
- 问题模型
  - 路线是一个带权图，图中各边的费用（权）为正数
  - 图的一条周游路线是包括V 中的每个顶点在内的一条回路，周游路线的费用是**这条路线上所有边的费用之和**
  - 旅行售货员问题的解空间可以组织成一棵树，从树的根结点到任一叶结点的路径定义了图的一条周游路线——旅行售货员问题要在图G 中找出**费用最小的周游路线**

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

范例7–旅行售货员问题

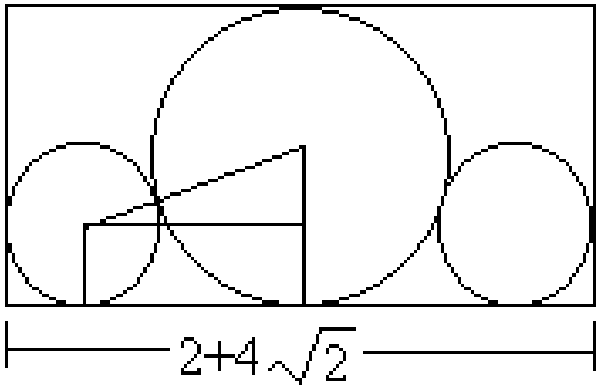
```
template<class Type>
void Traveling<Type>::Backtrack(int i)
{ //获得一个排列
  if (i == n) {
    if (a[x[n-1]][x[n]] != NoEdge && a[x[n]][1] != NoEdge &&
      (cc + a[x[n-1]][x[n]] + a[x[n]][1] < bestc || bestc == NoEdge)) {
      for (int j = 1; j <= n; j++) bestx[j] = x[j];
      bestc = cc + a[x[n-1]][x[n]] + a[x[n]][1];
    }
  }
  else { //继续递归
    for (int j = i; j <= n; j++)
      // 是否可进入x[j]子树?
      if (a[x[i-1]][x[j]] != NoEdge &&
        (cc + a[x[i-1]][x[i]] < bestc || bestc == NoEdge)) { // 可能有更优解，搜索子树
        Swap(x[i], x[j]);
        cc += a[x[i-1]][x[i]];
        Backtrack(i+1);
        cc -= a[x[i-1]][x[i]];
        Swap(x[i], x[j]);}
      }
  }
}
```

复杂度分析

算法backtrack在最坏情况下可能需要更新当前最优解O((n-1)!)次，每次更新bestx需计算时间O(n)，从而整个算法的计算时间复杂性为O(n!)。

范例8–圆排列问题

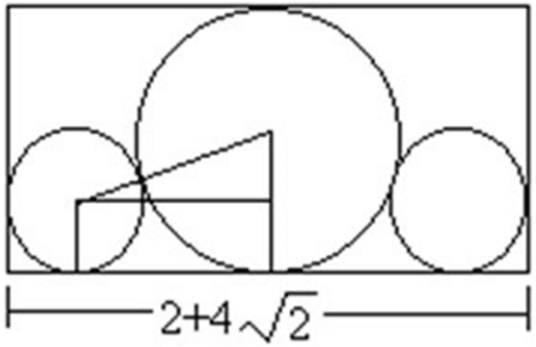
- 问题描述
  - 给定 n 个大小不等的圆 $c_1, c_2, \ldots, c_n$，现要将这 n 个圆排进一个矩形框中，且要求各圆与矩形框的底边相切
  - 圆排列问题要求从 n 个圆的所有排列中找出有最小长度的圆排列
- 示例
  - 当 n=3，且所给的 3 个圆的半径分别为 1，1，2 时，这 3 个圆的最小长度的圆排列如图所示，其最小长度为 $2+4\sqrt{2}$



華中科技大學 人工智能与自动化学院

范例8–圆排列问题

```
void Circle::Backtrack(int t)
 {
   if (t>n) Compute();
   else
     for (int j = t; j <= n; j++) {
        Swap(r[t], r[j]);
        float centerx=Center(t);
        if (centerx+r[t]+r[1]<min) { //下界约束
           x[t]=centerx;
           Backtrack(t+1);
        }
       Swap(r[t], r[j]);
     }
}
```




```
float Circle::Center(int t)
{ // 计算当前所选择圆的圆心横坐标
   float temp=0;
   for (int j=1; j<t;j++) { //j扫描前面所有圆
      float valuex=x[j]+2.0*sqrt(r[t]*r[j]);
      if (valuex>temp) temp=valuex;
   }
   return temp;
}

void Circle::Compute(void)
{ // 计算当前圆排列的长度
   float low=0, high=0;
   for (int i=1;i<=n;i++) {
     if (x[i]-r[i]<low) low=x[i]-r[i]; // 前端点
     if (x[i]+r[i]>high) high=x[i]+r[i]; // 后端点
   }
   if (high-low<min)
      min=high-low;
}
```


華中科技大學
人工智能与自动化学院

范例8–圆排列问题

- 复杂度分析
  - 由于算法backtrack在最坏情况下可能需要计算 O(n!)次当前圆排列长度，每次计算需 O(n)计算时间，从而整个算法的计算时间复杂性为 O((n+1)!)
  - 圆排列问题要求从 n 个圆的所有排列中找出有最小长度的圆排列
- 改进的地方
  - 1,2,…,n-1,n和 n,n-1, …,2,1这种互为镜像的排列具有相同的圆排列长度，只计算一个就够了，可减少约一半的计算量
  - 如果所给的 n个圆中有 k个圆有相同的半径，则这k个圆产生的 k! 个完全相同的圆排列，只计算一个就够了

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

范例9–连续邮资问题

- 问题描述
  - 假设国家发行了n种不同面值的邮票，并且规定每张信封上最多只允许贴m张邮票。
  - 连续邮资问题要求对于给定的n和m的值，给出邮票面值的最佳设计，在1张信封上可贴出从邮资1开始，增量为1的最大连续邮资区间。

例如，当n=5和m=4时，面值为(1, 3, 11, 15, 32)的5种邮票可以贴出邮资的最大连续邮资区间是1到70。

華中科技大學 人工智能与自动化学院

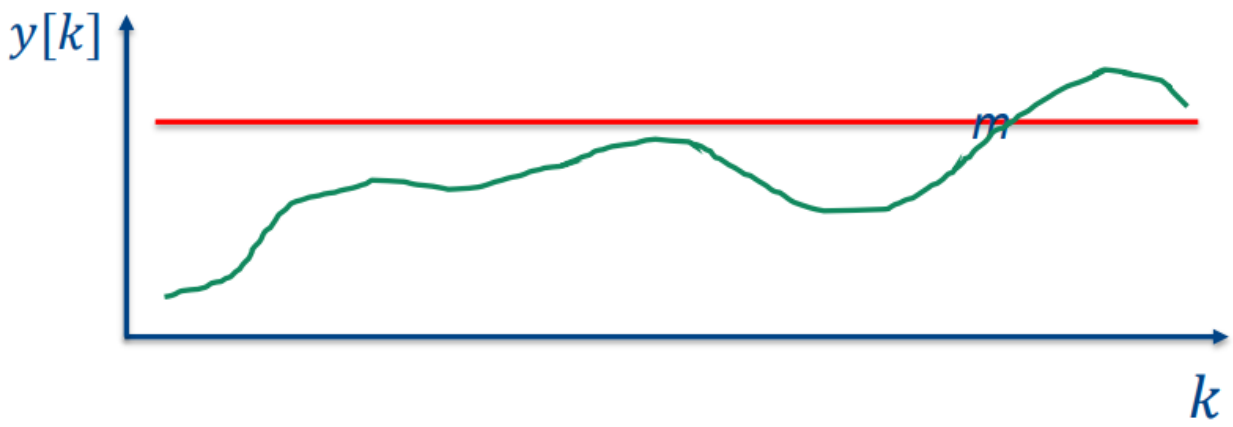
范例9–连续邮资问题

- 解向量
  - 用n元组x[1:n]表示n种不同的邮票面值，并**约定它们从小到大排列**。
  - **x[1]=1是唯一的选择**，此时最大连续邮资区间是[1:m]。
  - x[2]的取值范围是[2 : m+1]。
- 可行性约束函数
  - 已选定x[1:i-1]，最大连续邮资区间是[1: r]。
  - 接下来x[i]的可取值范围是[x[i-1]+1 : r+1]。
  - **连续约束下最大化r**
- 在用回溯法解连续邮资问题时可以用树表示解空间。树中各结点的度随x的不同取值而变化。

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

范例9–连续邮资问题

- **如何确定r的值？**
  - 计算X[1:i]的最大连续邮资区间在本算法中被频繁使用到，因此势必要找到一个高效的方法。
  - 考虑到直接递归的求解复杂度太高，我们不妨尝试计算用不超过m张面值为x[1:i]的邮票贴出邮资k所需的最少邮票数y[k]。
  - 通过y[k]可以很快推出r的值。

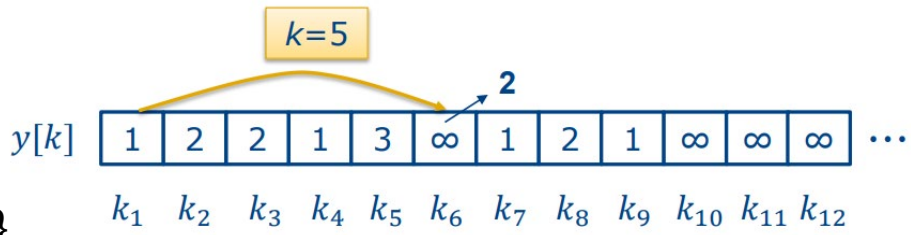


華中科技大學 人工智能与自动化学院

范例9–连续邮资问题

- **y[k]可以通过递推在O(n)时间内解决**

$$\begin{cases} y_i[k] = \min_{1\le t\le m}\{t + y_{i-1}[k - t * x_i] \\ y_1[k] = k \\ r_i = \min\{k | y_i[k] \le m, y_i[k+1] > m\} \end{cases}$$




```
for (int j=0; j<= x[i-2]*(m-1); j++)  //确定邮资更新范围
    if (y[j]<m)                       //从y_{i-1}[j]更新到y_i[j]
      for (int k=1; k<=m-y[j]; k++)   // k张x_i邮票
        if (y[j]+k < y[j+x[i-1]*k]) y[j+x[i-1]*k]=y[j]+k;
while (y[r]<maxint) r++;  //y[r]能够找到对应的连续邮资
```

回溯法效率分析

- 通过前面具体实例的讨论容易看出，回溯算法的效率在很大程度上依赖于以下因素：

  (1)产生x[k]的时间；

  (2)满足显约束的x[k]值的个数；

  (3)计算约束函数constraint的时间；

  (4)计算上界函数bound的时间；

  (5)满足约束函数和上界函数约束的所有x[k]的个数。

- 好的约束函数能显著地减少所生成的结点数。但这样的约束函数往往计算量较大。因此，在选择约束函数时通常存在生成结点数与约束函数计算量之间的折衷。

華中科技大學 人工智能与自动化学院

End of Course